

- I) Explique porque os dois ramos de um bloco residual não são correlacionados. Mostre que a variância da soma de duas variáveis aleatórias não correlacionadas é igual a soma das variâncias.
- II) Por que precisamos usar *batch norm* quando usamos *skip connections*?
- III) Quantos parâmetros treináveis uma camada de *batch norm* possui?
- IV) O grafo computacional do *batch norm* é dado pela Figura 1. Nessa figura, os círculos marcam cada uma das funções que são executadas durante o processo de um forward que aplica *batch norm*. Tendo em vista esse contexto, faça o que é solicitado nas questões abaixo:

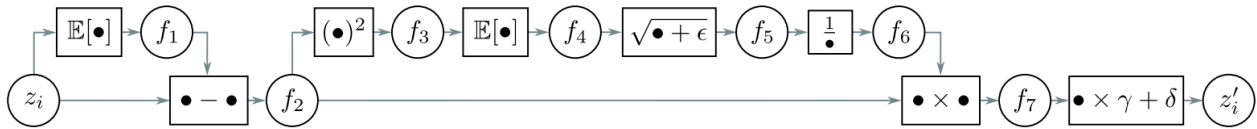


Figura 1: Grafo Computacional Batch Norm

As equações abaixo detalham o grafo computacional:

$$f_1 = \mathbb{E}[z_i] \quad f_{2i} = x_i - f_1 \quad f_{3i} = f_{2i}^2 \quad f_4 = \mathbb{E}[f_{3i}]$$

$$f_5 = \sqrt{f_4 + \epsilon} \quad f_6 = \frac{1}{f_5} \quad f_{7i} = f_{2i} \times f_6 \quad z'_i = f_{7i} \times \gamma + \delta$$

- a) Escreva código em Python para implementar o *forward pass*.
- b) Descubra as derivadas de cada nodo do grafo.
- c) Crie uma expressão que computa $\frac{\partial z'_i}{\partial z_i}$
- d) Implemente o *backward pass* em Python.
- V) Calcule a saída do caminho $F(x)$ e a saída final de um bloco residual onde a entrada e a saída possuem as mesmas dimensões. Lembre que:

$$H(x) = F(x) + x \quad (1)$$

Onde $F(x)$ representa uma camada convolucional utilizando o Kernel K_1 . Assuma padding = 1; stride = 1; bias = 0.

$$X = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix} \quad K_1 = \begin{bmatrix} 0 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

- VI) Sobre a questão anterior, que aconteceria com a saída $H(x)$ se o kernel K_1 fosse treinado para ter todos os seus 9 pesos iguais a 0? O que isso significa para o treinamento de redes muito profundas?
- VII) Em redes neurais muito profundas, os gradientes podem "desaparecer" (ficando próximos de zero), impedindo que as primeiras camadas da rede aprendam. Qual é a "chave" para resolver o problema do vanishing gradient?

Gabarito

- I) Como um dos ramos de uma conexão residual passa por uma multiplicação por um vetor de pesos seguido de uma função de ativação não-linear, as correlações lineares são perdidas.
- II) Para não causar explosão das ativações nem dos gradientes devido ao aumento da variância.
- III) 2γ e δ
- IV) Ver aqui
- V) Cálculo de $F(x)$ (Convolução de x com K_1):

O kernel K_1 é um filtro simples que multiplica o pixel central por -1 e todos os vizinhos por 0 . Ao passarmos K_1 sobre a matriz de entrada com padding, o resultado de cada operação é o valor do pixel central da entrada multiplicado por -1 .

Portanto, a matriz $F(x)$ é a negação da matriz de entrada x :

$$F(x) = \begin{bmatrix} -1 & -2 & -3 \\ -4 & -5 & -6 \\ -7 & -8 & -9 \end{bmatrix}$$

Cálculo $H(x)$: A saída final do bloco residual é a soma do resultado $F(x)$ com a entrada original x .

$$H(x) = F(x) + x$$

$$H(x) = \begin{bmatrix} -1 & -2 & -3 \\ -4 & -5 & -6 \\ -7 & -8 & -9 \end{bmatrix} + \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

$$H(x) = \begin{bmatrix} (-1+1) & (-2+2) & (-3+3) \\ (-4+4) & (-5+5) & (-6+6) \\ (-7+7) & (-8+8) & (-9+9) \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

Resultado Final: A saída do bloco residual é uma matriz nula:

$$H(x) = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

- VI) Se o kernel K_1 fosse treinado para ser uma matriz de zeros, e assumindo que o bias B_1 também seja zero, o resultado da convolução $F(x)$ seria uma matriz inteiramente nula. A Saída final do bloco, $H(x)$, seria calculada como:

$$H(x) = F(x) + x = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} + x = x$$

O bloco se tornaria, efetivamente, uma transformação identidade, simplesmente passando a entrada x adiante sem qualquer alteração.

- VII) A "chave" são as conexões residuais (*skip connections*), introduzidas pelas ResNets. Elas criam um "atalho" que soma a entrada (x) diretamente à saída de um bloco de camadas ($F(x)$). Garantindo que o gradiente sempre tenha um caminho direto (o atalho $+x$) para fluir, impedindo que ele se multiplique até desaparecer.